Automatic summarization system

Albert Sirvent Jerez, Enrique Cortés Tárraga and Carlos Sanchis Server

February 26th, 2026

Contents

1 Introduction **3**

2 Data **3**
2.1 Extracted queries . . . 3
2.2 Structured output . . . 5

3 Architecture **5**
3.1 Component details . . . 5
3.1.1 Stage 1: PDF Preprocessing . . . 5
3.1.2 Stage 1.5: Optional Analysis . . . 6
3.1.3 Stage 2: NLU Data Extraction . . . 6
3.1.4 Stage 3: Summary Generation . . . 7
3.2 Technology Stack Summary . . . 7
3.3 Data Transformation Summary . . . 7
3.4 Architecture Diagram . . . 8

4 Evaluation **9**
4.1 Model Election . . . 9
4.2 Prompt Used . . . 10
4.3 Model Evaluation . . . 10
4.3.1 Strengths . . . 11
4.3.2 Limitations and Areas for Improvement . . . 11

5 Conclusion **11**

1 Introduction

The density of legal language and the lack of a standard structured format in official public administration documents can pose a significant barrier for people who read them seeking specific information. That is the case of students who consult the Boletín Oficial del Estado (BOE) in search of information about scholarships.

The objective of this project is to help them by programming an automatic summarization system: an end-to-end NLP pipeline that combines information extraction techniques with a generative text model and is able to identify and classify critical data points.

During the development of this project, we avoided the use of commercial LLMs such as GPT, Llama or Gemini because we wanted to apply the NLP techniques and the knowledge acquired in class. Throughout this report we will indicate what techniques and models we have applied at each point and the reason for choosing them.
Source code: `https://github.com/minineng/MasterUA-APLN`

2 Data

The corpus for this project consists of five official scholarship documents in PDF format published in the Boletín Oficial del Estado (BOE), covering the academic years from 2021-22 to 2025-26. These documents were provided by the professors and were preprocessed into Markdown files before attempting to extract the data.

To transform these dense documents into structured data, we developed an extraction schema using a combination of Extractive Question Answering (QA) and Named Entity Recognition (NER). As we will see in the next section, this data extraction is managed by the `extractor.py` module, which uses two specialized language models assigned to specific tasks:

- **deepset/roberta-base-squad2 (QA Model)**
  Responsible for the core extraction logic. By processing the cleaned Markdown text, it answers a set of predefined natural language queries, written in Spanish to match the original language. These questions are detailed in the next section.

- **es_core_news_lg (NER Model)**
  The es_core_news_lg (our NER) model from spaCy is used for identification of atomic entities such as dates, organizations and specific amounts. This is a large model trained on official news and media corpora, and acts as a validation layer, ensuring that the segments identified by the QA model correspond to the expected entity types.

2.1 Extracted queries

The questions we made to the QA model were focused on extracting all the information that we considered relevant for having a complete view on the grant.

The JSON file that has these questions can be found here:
`https://github.com/minineng/MasterUA-APLN/tree/main/data/config/questions.json`

- **año_academico:** ¿A qué curso académico pertenece esta convocatoria?

- **enseñanzas_incluidas:** ¿Qué enseñanzas o estudios están incluidos en esta convocatoria de becas?

- **ejercicio_renta_computable:** ¿Qué ejercicio fiscal o año de ingresos se tiene en cuenta para el cálculo de la renta familiar?
- **fecha_limite:** ¿Cuál es la fecha límite para presentar la solicitud?
- **cobertura_matricula:** ¿Qué cubre la beca de matrícula y a qué estudios se aplica?
- **requisitos_academicos:** ¿Cuáles son los requisitos académicos generales para obtener la beca (matrícula, créditos o carga lectiva mínima)?
- **cuantia_beca_basica:** ¿Cuál es la cuantía de la beca básica?
- **cuantia_renta:** ¿Cuál es la cuantía de la beca fija ligada a la renta?
- **cuantia_variable_minima:** ¿Cuál es la cuantía mínima de la parte variable de la beca?
- **cuantia_residencia:** ¿Cuál es la cuantía de la beca fija ligada a la residencia?
- **nota_media_minima:** ¿Cuál es la nota requerida para las cuantías de excelencia?
- **excelencia_academica:** ¿Cuál es la cuantía de la beca fija ligada a la excelencia?
- **cuantia_excelencia_8_00_8_49:** ¿Cuál es la cuantía por excelencia académica para nota media entre 8,00 y 8,49?
- **cuantia_excelencia_8_50_8_99:** ¿Cuál es la cuantía por excelencia académica para nota media entre 8,50 y 8,99?
- **cuantia_excelencia_9_00_9_49:** ¿Cuál es la cuantía por excelencia académica para nota media entre 9,00 y 9,49?
- **cuantia_excelencia_9_50_o_mas:** ¿Cuál es la cuantía por excelencia académica para nota media igual o superior a 9,50?
- **umbral_matricula:** ¿Qué umbral de renta se exige para obtener la beca de matrícula?
- **umbral_renta:** ¿Qué umbral de renta se exige para obtener la beca fija ligada a la renta?
- **umbral_residencia:** ¿Qué umbral de renta se exige para obtener la beca fija ligada a la residencia?
- **umbral_variable:** ¿Qué umbral de renta se exige para obtener la parte variable?
- **compensacion_insularidad:** ¿Existe compensación por insularidad y cuáles son sus cuantías?
- **incremento_discapacidad:** ¿Cómo se incrementan las cuantías de la beca en caso de discapacidad y en qué porcentajes?
- **exencion_victimas:** ¿Qué medidas especiales se aplican a víctimas de violencia de género o violencia sexual?
- **como_solicitar:** ¿Cómo y dónde se presenta la solicitud de beca?
- **plazo_solicitud_inicio:** ¿Cuál es la fecha de inicio del plazo de solicitud?
- **plazo_solicitud_fin:** ¿Cuál es la fecha de fin del plazo de solicitud?

2.2 Structured output

The output data is saved in two different formats: CSV and JSON (but eventually we decided to use the second one to manage the summarization):

- **CSV Format:** `https://github.com/minineng/MasterUA-APLN/tree/main/data/processed/final_scholarship_dataset.csv`
- **JSON format:** `https://github.com/minineng/MasterUA-APLN/tree/main/data/processed/final_scholarship_dataset.json`

As a result, we obtained a large set of variables per document, ensuring every important point is represented. Those variables cover administrative data (document title, academic year, official publication dates), specific amounts for scholarships related to residence, income thresholds and academic excellence, and temporal constraints (start and end dates for the application period).

3 Architecture

The system is organized as a modular NLP pipeline with two main stages: information extraction and summary generation. In the first stage, the original PDF documents are converted into cleaned Markdown and processed with QA and NER models to populate a structured JSON representation. In the second stage, that JSON representation is passed to an instruction-tuned generative model that produces the final natural-language summary.

The system is organized into two main pipelines that work sequentially:

1. Information Extraction Pipeline (Stages 1 and 2): Transforms raw PDF documents into structured JSON data
2. Text Generation Pipeline (Stage 3): Converts structured data into human-readable summaries

The coupling point between both pipelines is the structured JSON dataset containing extracted scholarship information.

3.1 Component details

3.1.1 Stage 1: PDF Preprocessing

Module: ‘utils/preprocessing.py‘
Purpose: Convert PDF documents into clean, structured Markdown text suitable for NLP processing.

Component	Technology	Description
PDF Conversion	‘pymupdf4llm‘	Converts PDF to Markdown while preserving document structure
Text Cleaning	Custom regex	Removes page numbers, headers, footers, and irrelevant sections

Input: ‘data/corpus/*.pdf‘
Output: ‘data/preprocessed/*.clean.md‘

3.1.2 Stage 1.5: Optional Analysis

Module: ‘utils/analysis.py‘
Purpose: Perform TF-IDF term scoring analysis and generate word clouds for insights into document content. This stage is optional and provides additional analytical value but is not required for the main pipeline.

Component	Technology	Description
Tokenization	SpaCy	Custom tokenizer with Spanish stopwords filtering
Term Scoring	TfidfVectorizer	Computes TF-IDF scores for each term per document
Visualization	WordCloud	Generates PNG word clouds from top-scoring terms

Process:

1. Normalize text using the Extractor’s tokenizer
2. Build TF-IDF matrix across all documents
3. Extract top terms per document by score
4. Generate word cloud images for visual inspection

Input: ‘data/preprocessed/*.clean.md‘
Output:

- `‘data/analysis/*_analysis.json‘ - Term scores per document`
- `‘data/analysis/*_wordcloud.png‘ - Visual word cloud images`

3.1.3 Stage 2: NLU Data Extraction

Module: ‘utils/extractor.py‘
Purpose: Extract structured scholarship information using Named Entity Recognition and Question Answering models.

Named Entity Recognition (NER)

Component	Technology	Description
Tokenization	SpaCy	Custom tokenizer with Spanish stopwords filtering
Term Scoring	TfidfVectorizer	Computes TF-IDF scores for each term per document
Visualization	WordCloud	Generates PNG word clouds from top-scoring terms

Question Answering (QA)

Component	Technology	Description
Model	‘mrm8488/ bert-base-spanish-wwm-cased-finetuned-spa-squad2-es‘	Spanish QA model
Task	Extractive QA	Answer 27 predefined questions about scholarships
Configuration	‘data/config/questions.json‘	Question templates

Extracted Fields:

- Academic year, target studies, deadlines
- Scholarship amounts (basic, income-linked, residence, variable, excellence)
- Income thresholds
- Special conditions (disability, insularity, victims)

Input: ‘data/preprocessed/*.clean.md‘ + ‘data/config/questions.json‘
Output: Structured dictionary with 29 fields per document

3.1.4 Stage 3: Summary Generation

Module: ‘utils/summarization.py‘
Purpose: Generate human-readable article-style summaries from structured JSON data.

Component	Technology	Description
Model	‘Qwen/Qwen2.5-3B-Instruct‘	Multilingual instruction-tuned LLM
Prompt	‘data/config/summary_generation_prompt.txt‘	Instructions for article-style generation

Prompt Design:

- Role: Public policy writer
- Format: 8-15 line article in Spanish
- Requirements: Cover all key fields without hallucination
- Constraints: No JSON mention, no bullet points

Input: ‘data/processed/final_scholarship_dataset.json‘
Output: ‘data/summary/*.summary.txt‘

3.2 Technology Stack Summary

Layer	Technologies
PDF Processing	PyMuPDF, pymupdf4llm
NLP - NER	SpaCy (es_core_news_sm)
NLP - QA	Hugging Face Transformers, BERT (Spanish fine-tuned)
Text Generation	Qwen2.5-3B-Instruct, PyTorch
Data Processing	Pandas, scikit-learn (TF-IDF)
Configuration	JSON-based question templates, text prompts

3.3 Data Transformation Summary

Stage	Input Format	Output Format	Key Transformation
Preprocessing	PDF	Markdown	Document structure extraction
Extraction	Markdown	Dictionary	Unstructured → Structured
Generation	JSON	Plain Text	Structured → Natural Language

3.4 Architecture Diagram

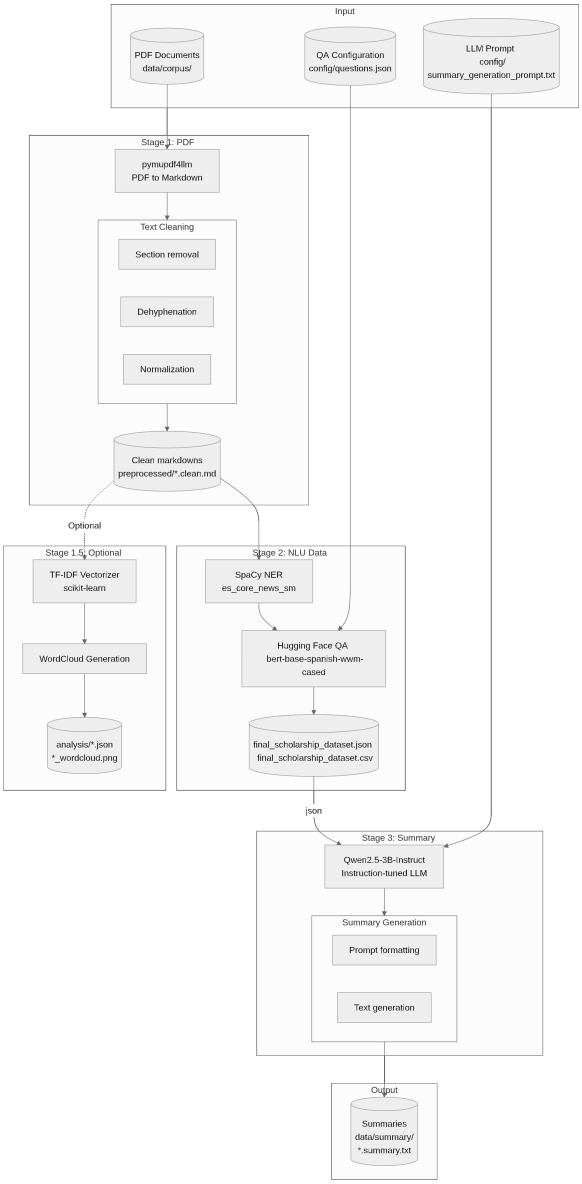

Figure 1: Architecture Diagram

4 Evaluation

In this stage is where we use the information extracted in the previous stage to generate a summary of each document, trying to capture all the information possible and focus it in a clear way. During the development, we decided to separate this stage from the initial one, so to be able to run it independently, as the information extraction is a very time-consuming process.

4.1 Model Election

The first step of this stage is to choose the language generation model that we will use to generate the summaries. We were thinking on using Ollama to generate the summaries but we tried to keep everything "within" Hugging-Face. Our first choices were focused on lightweight models only trained in spanish, these were the ones we tested (with the issues we found):

- **SEBIS/legal_t5_small_summ_es**
  Objective: Small T5 model fine-tuned for legal text summarization in Spanish.
  Issue encountered: Not instruction-following and unable to reliably interpret structured JSON inputs.

- **mrm8488/bert2bert_shared-spanish-finetuned-summarization**
  Objective: Encoder-decoder model based on BETO fine-tuned for Spanish news summarization (MLSUM).
  Issue encountered: Technical instability on macOS (meta tensor errors) and poor adaptation to structured dictionary inputs.

- **Narrativa/bsc_roberta2roberta_shared-spanish-finetuned-mlsum-summarization**
  Objective: RoBERTa2RoBERTa model trained for abstractive news summarization in Spanish.
  Issue encountered: Designed for continuous narrative text rather than structured JSON data.

After having little success with the previous models, we decided to try multilingual models (with spanish support) and we also removed our requirement to have it lightweight. We thought on using instruction-tuned, as they work better for transforming structured JSON inputs into coherent narrative text (they are explicitly trained to interpret task-specific instructions rather than summarize).

- **google/flan-t5-small**
  Objective: Instruction-tuned multilingual model capable of following prompts and generating Spanish text.
  Issue encountered: Limited coherence and coverage when handling long or complex JSON inputs.

- **Qwen/Qwen2.5-3B-Instruct (Selected Model)**
  Objective: Multilingual instruction-following model optimized for structured input understanding and coherent text generation.
  Issue encountered: Higher computational requirements compared to smaller summarization models.

Finally, we decided to use the Qwen2.5-3B-Instruct model, as it was the one that generated the best summaries and was able to understand better the structured input without being "too big" or too slow.

4.2 Prompt Used

The generation stage relies on a single instruction-based prompt designed to transform the structured JSON output into a fluent, article-style text in Spanish. The model is explicitly instructed to behave as a public policy writer and to produce a coherent narrative description of the scholarship without mentioning the JSON structure or using bullet points.

The prompt enforces coverage of key elements (academic year, target studies, financial components, income thresholds, deadlines, and special conditions) while explicitly prohibiting hallucinations or the addition of information not present in the input. This design ensures that the generated text remains faithful to the extracted structured data while adopting a natural journalistic style.

The exact prompt used in the system is shown below:

```
Eres un redactor especializado en convocatorias públicas.
A partir del siguiente JSON, redacta un texto informativo en español
como si fuera un artículo explicando en qué consiste la beca.

El texto debe:
- Explicar el curso académico al que corresponde.
- Indicar a qué enseñanzas o estudios se dirige.
- Describir las cuantías principales (renta, residencia, básica,
  variable, excelencia si existe).
- Mencionar los umbrales de renta si aparecen.
- Explicar los plazos de solicitud.
- Incluir condiciones especiales (discapacidad, insularidad, víctimas)
  si están presentes.

No inventes información.
No añadas datos que no estén en el JSON.
No menciones el JSON ni uses formato estructurado.
Redacta un texto fluido de entre 8 y 15 líneas.

JSON:
{json_text}

Artículo:
```

4.3 Model Evaluation

The generation stage was evaluated using the structured JSON dataset as input and comparing the produced summaries across all academic years. The main differences observed between the tested models were related to their ability to interpret structured JSON inputs and transform them into coherent narrative text, as well as their stability in a local environment.

Finally, the selected model (Qwen2.5-3B-Instruct) demonstrated solid instruction-following capabilities and good narrative quality. The main caveat was generation length, as we had several issues with truncation when the generated text was too long, so we had to adjust the generation parameters to ensure that the output remained within a reasonable length while still capturing all relevant information.

The main observations are summarized below.

4.3.1 Strengths

- Produces fluent, coherent article-style texts in Spanish.
- Correctly introduces the academic year and overall purpose of the scholarship.
- Integrates multiple financial components (income-linked, residence-linked, basic and variable amounts) into a unified narrative.
- Includes excellence-related bonuses and special conditions (e.g., disability increments) when clearly present in the JSON.
- Transforms structured numerical values into readable natural language without relying on bullet-point formatting.
- Maintains consistent tone and structure across all academic years.

4.3.2 Limitations and Areas for Improvement

- Propagates inconsistencies present in the extracted JSON (e.g., conflicting deadlines or imprecise values).
- Occasionally simplifies or slightly misinterprets categorical fields when the input is ambiguous.
- Does not perform logical validation across fields (e.g., checking temporal coherence between academic year and deadlines).
- Output length must be carefully controlled to avoid truncation in longer summaries.

5 Conclusion

This project shows that a classical NLP pipeline combined with a compact instruction-tuned language model can be effective for assisting students who need to understand complex official scholarship calls. The extraction stage makes the relevant information explicit and machine-readable, while the generation stage converts that structure into a more accessible narrative form.

In the generation stage, model selection proved crucial for producing coherent and faithful summaries. Lightweight Spanish summarization models were not suitable for structured JSON inputs, whereas instruction-tuned multilingual models performed significantly better.

The selected model, Qwen2.5-3B-Instruct, achieved a good balance between narrative quality and structured input understanding. However, generation quality remains strongly dependent on the accuracy of the information extraction stage, as inconsistencies in the extracted data are directly reflected in the final summaries.

Overall, the generation stage successfully produced coherent and informative texts aligned with the structured input.